UNIVERSIDAD NACIONAL DE LOJA
Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables

ARQUITECTURA DEL SISTEMA

Proyecto: Prototipo SmartPark

Desarrollo de un prototipo web para la clasificación automática del estado de ocupación de estacionamientos integrando el modelo YOLOv11

Proyecto de Integración Curricular

Elaborado por:	Saúl Alejandro Ludeña Ocampo
Director:	Ing. Edwin René Guamán Quinche
Fecha de revisión:	27 de abril del 2026
Aprobado por:	Ing. Edwin René Guamán Quinche
Fecha de aprobación:	07 de mayo del 2026

Edwin René Guamán Quinche

ASESOR

1. Introducción

1.1 Propósito

El presente documento describe la arquitectura del prototipo SmartPark. Su objetivo es comunicar la organización estructural del sistema —componentes, responsabilidades e interfaces— a los distintos interesados del proyecto, y servir como referencia técnica durante las fases de implementación y revisión. Los interesados del proyecto y sus respectivos intereses en la arquitectura se detallan en la Tabla 1.

1.2 Alcance

La descripción de arquitectura cubre los tres módulos principales del sistema: el Frontend (interfaz web), el Backend (servidor de inferencia) y el Motor de IA (modelos de visión artificial). No incluye aspectos de infraestructura de producción ni despliegue en la nube, ya que el prototipo opera en entorno local.

1.3 Norma de referencia

ISO/IEC/IEEE 42010:2022 — Systems, software and enterprise: Architecture description.

2. Descripción General del Sistema

2.1 Sistema de interés

El sistema de interés es el Prototipo SmartPark: una aplicación web que automatiza la clasificación del estado de ocupación de estacionamientos mediante visión artificial. El sistema recibe imágenes estáticas de estacionamientos, permite al usuario delimitar las plazas de estacionamiento como regiones de interés y aplica el modelo YOLOv11 para determinar si cada plaza está libre u ocupada.

2.2 Patrón arquitectónico

El sistema sigue el patrón cliente-servidor en tres capas. El cliente se comunica con el servidor mediante solicitudes HTTP y recibe las respuestas a través de una API REST. El Backend, a su vez, delega el procesamiento de imágenes y la inferencia al Motor de IA, utilizando las bibliotecas Torch y OpenCV.

2.3 Interesados

Tabla 1. Interesados del sistema

ID	Interesado	Interés en la arquitectura
SH-01	Usuario (AC1)	Comprender cómo se procesan sus imágenes y qué resultados obtendrá.
SH-02	Equipo de desarrollo	Guiar la implementación de componentes y sus responsabilidades.
SH-03	Director del proyecto	Verificar que la arquitectura satisface los requisitos del SRS.

3. Vista Lógica: Componentes y Responsabilidades

Esta vista describe la descomposición del sistema en sus tres módulos principales — Frontend, Backend y Motor de IA — y los subcomponentes que contiene cada uno. Además, aborda las preocupaciones de estructura interna, interfaces entre componentes y la integración con el servicio externo RoboFlow.

El diagrama de arquitectura del sistema se presenta a continuación:

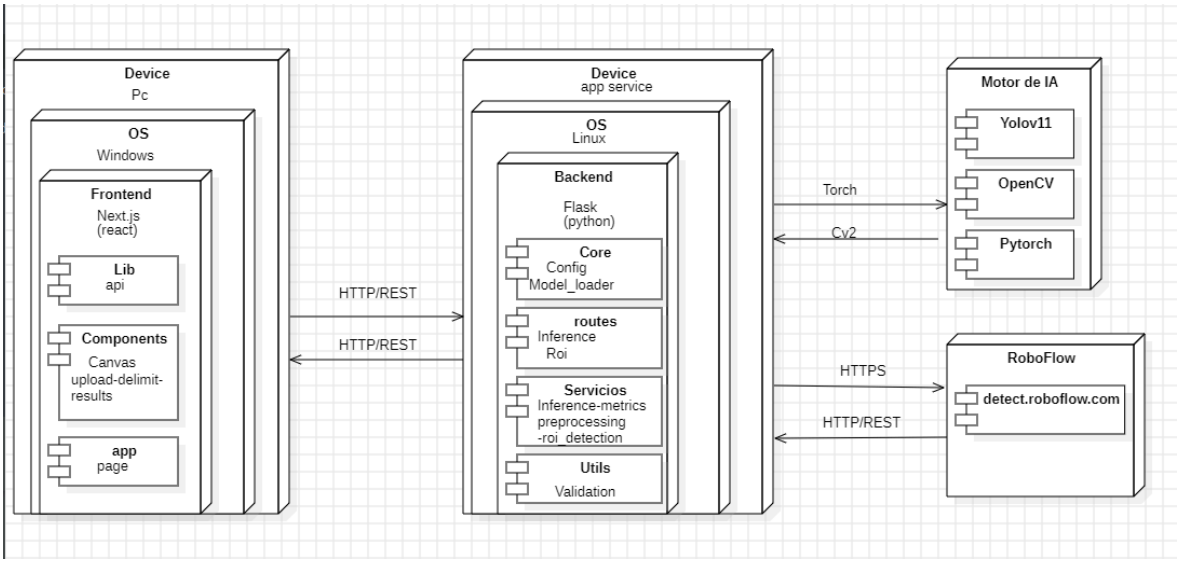

Figura 1: Diagrama de componentes del Prototipo SmartPark

3.1 Módulo Frontend

Tecnología base: Next.js. Responsable de la interacción con el usuario, la carga de imágenes, el trazado de ROI y la presentación de resultados.

Tabla 2. Subcomponentes del módulo Frontend

Subcomponente	RF asociado	Responsabilidad
Carga de imagen	RF1	Permite al usuario cargar imágenes mediante Drag & Drop o explorador de archivos. Valida el tipo y tamaño del archivo antes de enviarlo al backend.
Editor de ROI	RF2	Renderiza la imagen en un canvas interactivo sobre el que el usuario traza, edita y elimina polígonos que representan las plazas de estacionamiento.
Visualización	RF5 / RF6	Muestra sobre la imagen original un overlay semitransparente (verde = libre, rojo = ocupado) para cada ROI procesada, junto con el porcentaje de ocupación del estacionamiento.
AutoROI	RF2	Permite al usuario iniciar la detección automática de plazas de estacionamiento mediante el botón "Auto detectar plazas".

3.2 Módulo Backend

Tecnología base: Flask. Actúa como orquestador central, recibe las solicitudes del Frontend, coordina el rocesamiento de la imagen, lanza la inferencia y retorna los resultados.

Tabla 3. Subcomponentes del módulo Backend

Subcomponente	RF asociado	Responsabilidad
Controlador de imagen	RF1 / RF4	Recibe la solicitud HTTP del Frontend con la imagen y las coordenadas de las ROI. Coordina la ejecución del procesamiento y la inferencia, y construye la respuesta REST.
Servicio de procesamiento	RF3	Aplica la técnica de Letterboxing para redimensionar la imagen a 640×640 px, el algoritmo CLAHE para mejora de contraste y un filtro Gaussiano para reducción de ruido. Prepara la imagen para su entrada al modelo.
Servicio de inferencia	RF4	Invoca al modelo YOLOv11 sobre la imagen procesada. Calcula el centroide de cada bounding box detectado y determina el estado de ocupación de cada ROI según la presencia del centroide dentro de su polígono.
Servicio de métricas	RF6	Calcula las métricas de clasificación a partir de los resultados de la inferencia y genera el resumen estadístico de ocupación del estacionamiento.
Servicio de inferencia de ROIS	RF2	Identifica los límites de las plazas de estacionamiento mediante el modelo parking-space de RoboFlow.

3.3 Motor de IA

Tecnología base: PyTorch + OpenCV. Contiene modelos y algoritmos de visión artificial que ejecutan la detección y clasificación.

Tabla 4. Componentes del Motor de IA

Componente	Descripción
YOLOv11	Motor de inferencia principal. Implementado sobre Ultralytics. Detecta vehículos en la imagen y genera bounding boxes con score de confianza. Solo se consideran detecciones pertenecientes a las clases car, truck, bus y motorcycle del dataset COCO.
OpenCV	Biblioteca de visión artificial que ejecuta las transformaciones de procesamiento: redimensionamiento con Letterboxing, aplicación del algoritmo CLAHE y filtro Gaussiano.
PyTorch	Framework de aprendizaje profundo sobre el que se ejecuta el modelo YOLOv11. Gestiona la carga del modelo, la inferencia en CPU/GPU y la decodificación de las salidas del modelo.

3.4 Modelo de RoboFlow

Tecnología base: Roboflow Hosted API. Actúa como servicio externo de detección de plazas de estacionamiento.

Tabla 5. Componentes del Modelo de Roboflow

Componente	Descripción
Modelo parking-space	Modelo de detección de objetos disponible en Roboflow Universe, entrenado con 2.804 imágenes anotadas en dos clases correspondientes a plazas de estacionamiento. Es invocado mediante una solicitud HTTP POST al endpoint https://detect.roboflow.com/\{project\}/\{version\}, enviando la imagen codificada en base64 junto con los parámetros api_key y confidence.
API alojada de RoboFlow	Infraestructura de inferencia en la nube sobre la que se ejecuta el modelo pre entrenado. La comunicación se realiza mediante el protocolo HTTPS y requiere conexión a internet activa.

4. Vista de Proceso: Flujo Principal de Operación

Esta vista describe la secuencia de interacciones entre los módulos del sistema durante el flujo principal de uso, desde la carga de una imagen hasta la visualización de resultados. Se presenta en forma de descripción por pasos, complementada por el diagrama de secuencia correspondiente.

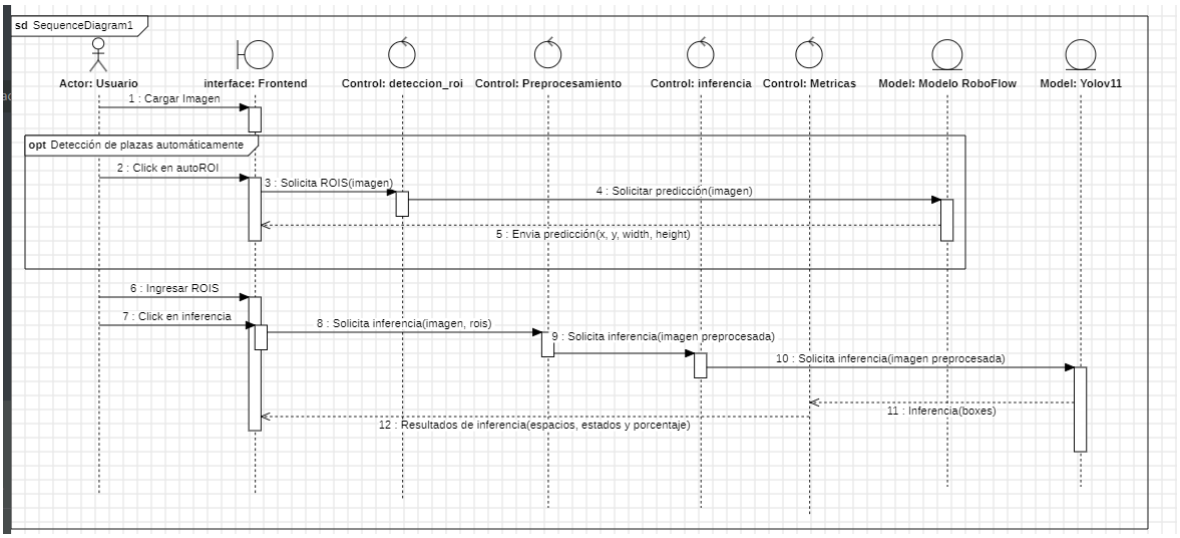

Figura 2: Flujo de operación principal del sistema

Tabla 6. Flujo principal de operación

Paso	Actor / Módulo	Acción
1	Usuario -> Frontend	El usuario carga una imagen de estacionamiento mediante la interfaz Drag & Drop.
2 [opt]	Usuario -> Frontend	El usuario hace clic en "Auto detectar plazas" para delimitar las plazas de forma automática.
3 [opt]	Frontend -> Control: deteccion_roi	El Frontend envía la imagen al controlador de detección de ROIs para iniciar el procesamiento automático.
4 [opt]	Control: deteccion_roi -> Modelo RoboFlow	El controlador solicita la predicción de plazas al modelo de RoboFlow.
5 [opt]	Modelo RoboFlow -> Control: deteccion_roi	RoboFlow retorna las coordenadas (x, y, width, height) de cada plaza detectada.
6	Usuario -> Frontend	El usuario ingresa manualmente las ROIs sobre el lienzo o confirma las detectadas automáticamente.
7	Usuario -> Frontend	El usuario hace clic en "Ejecutar Inferencia" para iniciar el análisis de ocupación.
8	Frontend -> Control: inferencia	El Frontend envía la imagen y las ROIs al controlador de inferencia del Backend mediante una solicitud HTTP POST.
9	Control: inferencia -> Control: procesamiento	El controlador de inferencia delega el procesamiento de la imagen al servicio correspondiente.

10	Control: preprocesamiento -> Motor YOLOv11	El servicio aplica Letterboxing, CLAHE y filtro Gaussiano, y pasa la imagen procesada al motor de detección.
11	Motor YOLOv11 -> Control: Inferencia	YOLOv11 ejecuta la inferencia y retorna los bounding boxes detectados.
12	Control: inferencia -> Frontend -> Usuario	El Backend retorna los resultados de inferencia (estados por espacio, porcentaje de ocupación y métricas). El Frontend renderiza el overlay de colores sobre la imagen y muestra el porcentaje de ocupación.

5. Vista de componentes: Diagrama de componentes

A continuación, se presenta el diagrama de componentes, el cual muestra la estructura interna del sistema y las interacciones entre sus componentes.

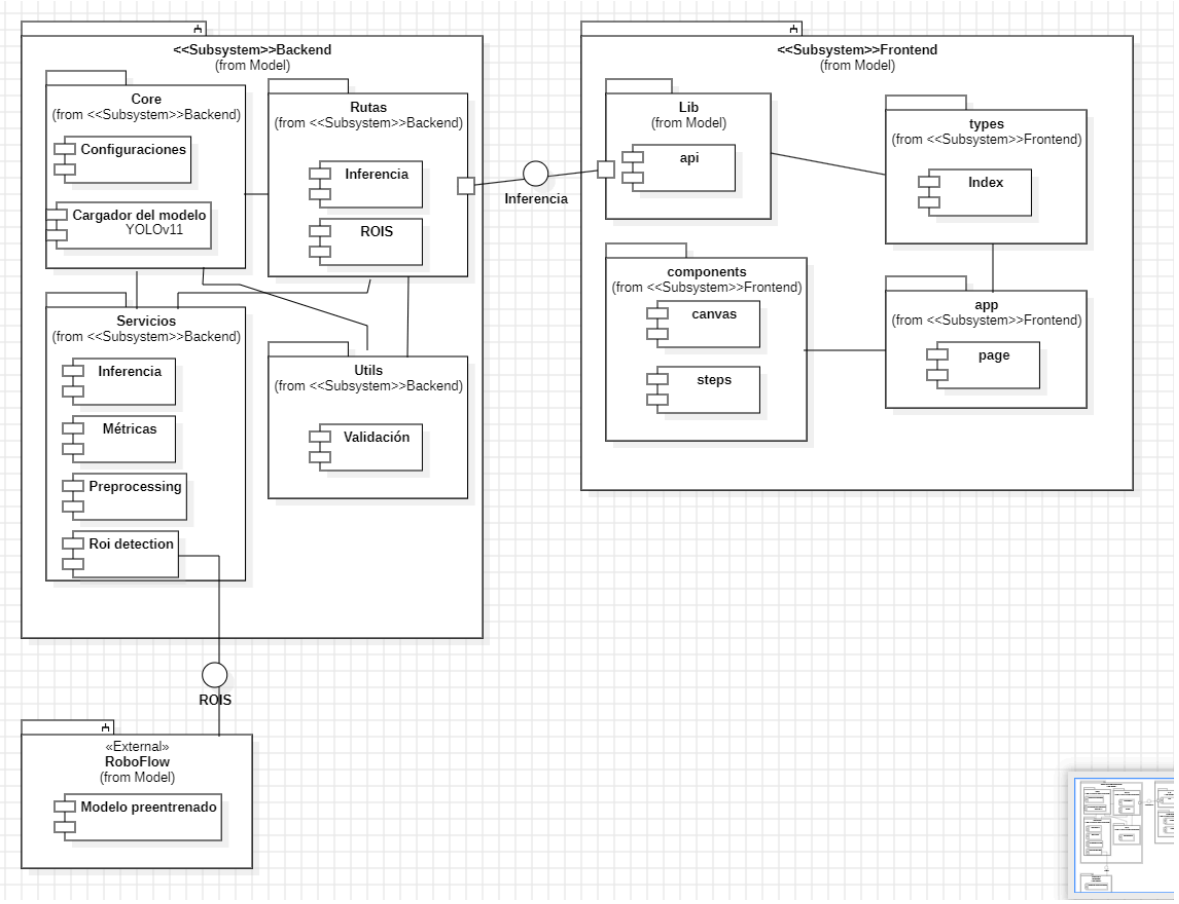

Figura 3: Diagrama de componentes

6. Interfaces entre Módulos

La siguiente tabla describe las interfaces de comunicación entre los módulos del sistema.

Tabla 7. Interfaces entre módulos

ID	Módulo origen	Módulo destino	Protocolo	Descripción
IF-01	Frontend	Backend	HTTP REST	Envío de la imagen y las coordenadas de las ROI trazadas por el usuario.
IF-02	Backend	Frontend	HTTP REST	Retorno de los estados de ocupación por ROI y del resumen estadístico.
IF-03	Backend	Motor de IA	Llamada a función en proceso mediante Torch	Invocación del modelo YOLOv11 sobre la imagen procesada.
IF-04	Motor de IA	Backend	Llamada a función en proceso mediante cv2	Retorno de los bounding boxes y scores de confianza detectados.
IF-05	Backend	Modelo RoboFlow	HTTP REST	Envío de la imagen codificada mediante una solicitud POST a https://detect.roboflow.com/{project}/{version}, incluyendo api_key y confidence. RoboFlow retorna un JSON con las coordenadas de la plaza.

7. Decisiones Arquitectónicas

Se registran a continuación las decisiones de diseño arquitectónico más relevantes del sistema.

ID	Decisión adoptada	Justificación
ADR-01	Separación en módulos independientes.	Permite la sustitución independiente de cada módulo sin afectar a los demás.
ADR-02	Uso de Flask como framework de backend	Framework ligero adecuado para prototipos de investigación. Permite exponer la API REST con mínima configuración y facilita la integración directa con las bibliotecas Python de visión artificial.
ADR-03	Uso de Next.js para el Frontend	Provee renderizado reactivo necesario para el canvas interactivo de dibujo de ROI y presentación del overlay de resultados.
ADR-04	Umbral de confianza por defecto de 0.45 en YOLOv11	Valor establecido experimentalmente que equilibra la tasa de falsos positivos y falsos negativos en el dataset de prueba PKLot.
ADR-05	Lógica de ocupación basada en centroide del bounding box	El centroide es robusto ante detecciones parciales de vehículos en los límites de una ROI. Simplifica el cálculo de intersección.

8. Trazabilidad con el SRS

La siguiente tabla establece la correspondencia entre los requisitos funcionales definidos en el documento de especificación de requisitos de software y los componentes arquitectónicos que los implementan.

ID RF	Nombre del requisito	Componente(s) que lo implementan
RF1	Gestión de carga de imágenes	Frontend: Carga de imagen → Backend: Controlador de imagen
RF2	Delimitación manual y automática de regiones de interés	Frontend: Editor de ROI → Backend: Servicio de inferencia de ROIs → RoboFlow (externo)
RF3	Procesamiento de imágenes	Backend: Servicio de procesamiento → Motor de IA: OpenCV
RF4	Análisis de ocupación mediante YOLOv11	Backend: Servicio de inferencia → Motor de IA: YOLOv11 + PyTorch
RF5	Visualización de resultados	Frontend: Visualización
RF6	Reporte de métricas de ocupación	Backend: Servicio de métricas → Frontend: Visualización